

Lab 6 Report

Ryan Wilson

Due: 03/10/2026 @ 11:00 AM

1 Introduction

The purpose of this lab report is to determine the time complexity of four numerical integration methods: the Midpoint Rule, Simpson's 1/3 Rule, Simpson's 3/8 Rule, and the Gauss Quadrature. To find the Big O complexity, analyzing the number of operations required for each method and then finding the growth rate is necessary.

2 Midpoint Rule

The midpoint of the function is calculated as: $f(x_{mp}) = f((a+b)/2)$

The function is calculated one time. Since the method has a fixed number of calculations regardless of the size of the function, the number of lines of code executed is constant. Therefore, the Big O complexity is $\mathcal{O}(1)$.

3 Simpson's 1/3 Rule

The Simpson's 1/3 Rule calculates the function at 3 different points: the left bound, the midpoint, and the right bound. Simpson's 1/3 Rule is calculated as: $f(x_{s,1/3}) = f(a) + 4f((a+b)/2) + f(b)$

The function is calculated one time. Since the method has a fixed number of calculations regardless of the size of the function, the number of lines of code executed is constant. Therefore, the Big O complexity is $\mathcal{O}(1)$.

4 Simpson's 3/8 Rule

The Simpson's 3/8 Rule calculates the function at 4 different points: the left bound, two internal points that are evenly spaced, and the right bound. Simpson's 3/8 Rule is calculated as: $f(x_{s,3/8}) = f(a) + 3f((2a+b)/3) + 3f((a+2b)/3) + f(b)$.

The function is calculated one time. Since the method has a fixed number of calculations regardless of the size of the function, the number of lines of code executed is constant. Therefore, the Big O complexity is $\mathcal{O}(1)$.

5 Gauss Quadrature

The Gauss Quadrature, or Gauss Quad, uses a loop to iterate through a selected number n of quad points. The code calculates the function for each value

of n , staying between the bounds of 0 to 10. Gauss Quad is calculated as: $x_{gq} = f(c + mt_i)$ where $c = (a + b)/2$, $m = (b - a)/2$, and t is the abscissa, or horizontal value.

The function uses a loop that runs from $i=1$ to n . Each loop iteration calculates a constant number of operations, meaning the total number of lines of code executed is proportional to n . Therefore, the Big O complexity is $\mathcal{O}(n)$.

6 Sources

- AERE361 Lab Manual 6
- TAs
- <https://pomax.github.io/bezierinfo/legendre-gauss.html>